FOSDEM’26

A Performance Comparison of Kubernetes Multi-Cluster Networking

<jlema@redhat.com> José Castillo Lema, Red Hat
<rsevilla@redhat.com> Raúl Sevilla Cañavate, Red Hat
2026-02-01

whoarewe

```
José Castillo Lema {
     performance-engineer;
     Red Hat, OpenShift perf&scale;
     location = “Madrid, Spain”;
};
```

```
Raúl Sevilla Canavete {
     performance-engineer;
     Red Hat, OpenShift perf&scale;
     location = “Madrid, Spain”;
};
```

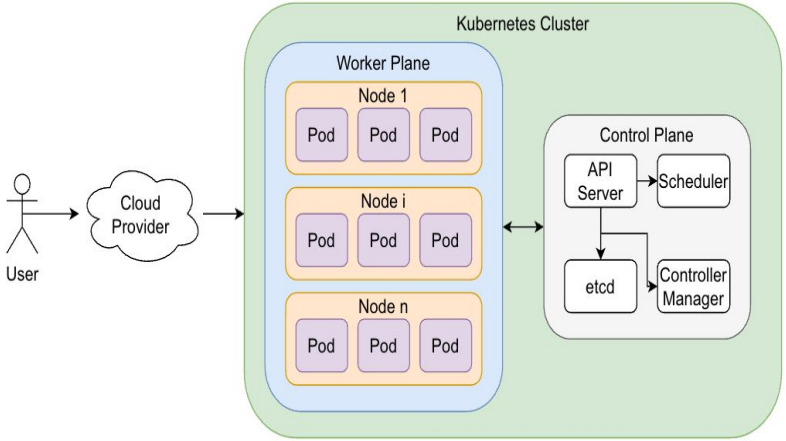
```
Sai Sindhur Malleni  {
     engineer-manager;
     Nvidia, DGX Cloud;
     location = “Boston, MA, USA”;
};
```

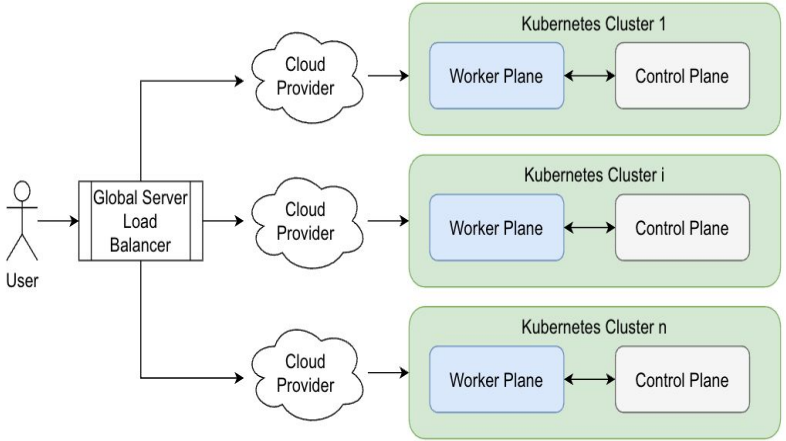
```
André Bauer {
     assistant-professor;
     Illinois Tech;
     location = “Chicago, IL, USA”;
};
```

Overview

- Introduction
  - Single-Cluster vs Multi-Cluster
- Multicluster networking solutions feature comparison
  - Submariner
  - Skupper
  - Istio
- Testbed description
- Tooling
- Results
- Summary
- Future work

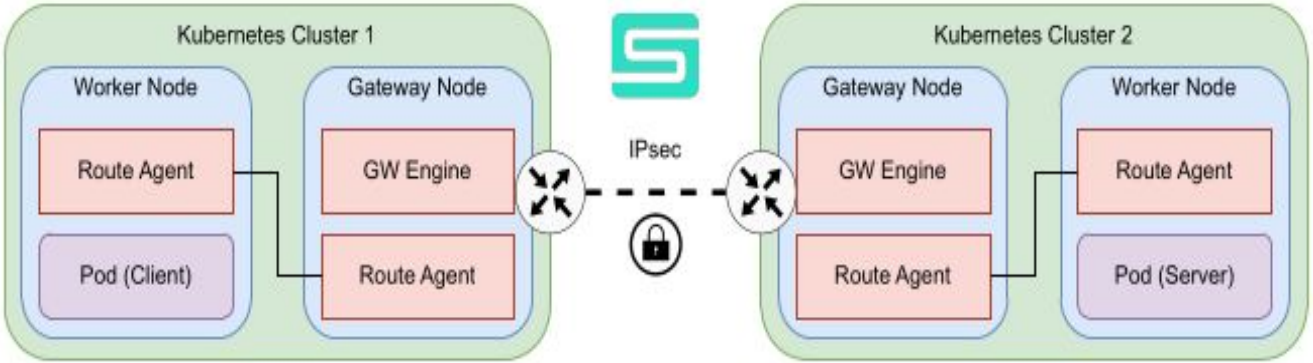
Single vs Multi-Cluster





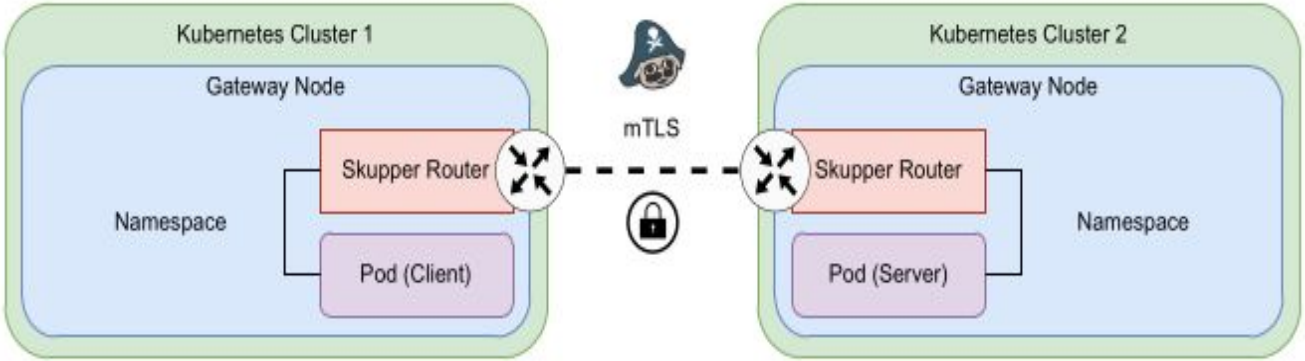
Why Multi-Cluster? Geo-redundancy, scale, fault-toleration

Submariner



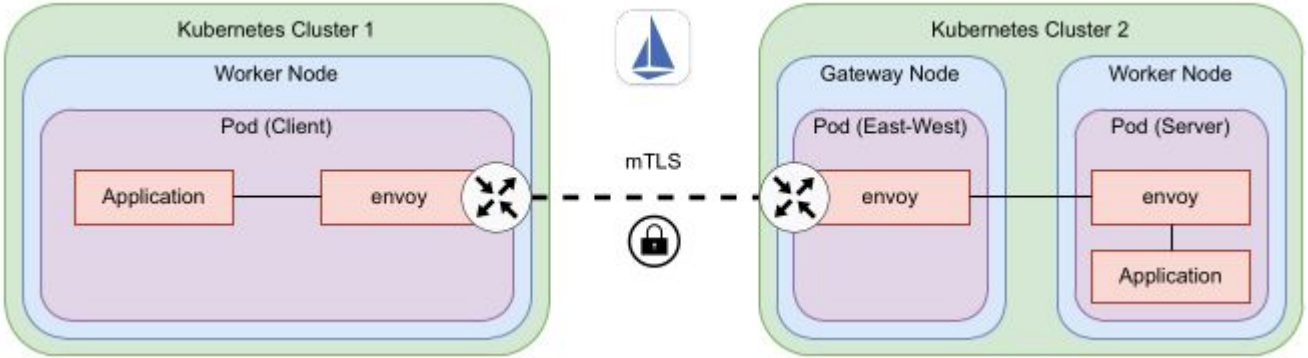
Cross-cluster **L3 connectivity** using encrypted or unencrypted connections, with the traffic going through the submariner gateway

Skupper



L7 service interconnect that enables secure communication across Kubernetes clusters with no VPNs or special firewall rules.

Istio multicluster: multi-primary

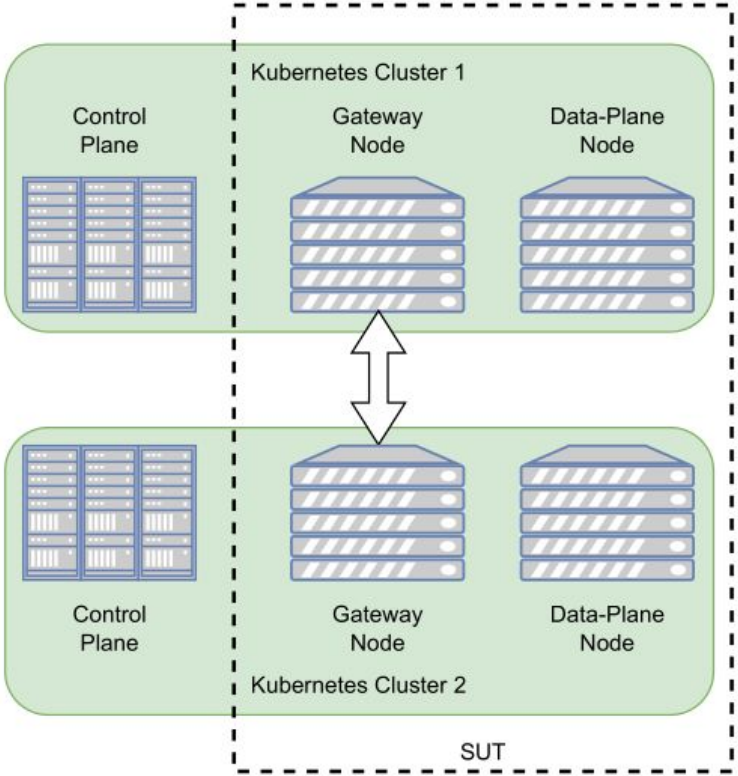


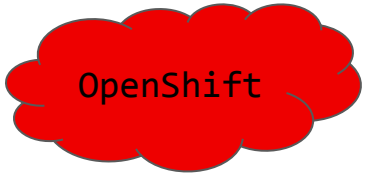
To enable **inter-cluster endpoint discovery**, trust needs to be established between each primary cluster in the mesh (in our case **using a intermediate CA signed by the same root CA**)

Feature comparison

	Skupper	Submariner	Istio
Operation layer	L4	L3	L4/L7
Network topology	Point to point	Hub/spoke	Network mesh
Inter-connect level	Namespace	Cluster	Service
Service discovery	✓	✓	✓
Security features	mTLS	IPsec	mTLS
Authentication policies	Limited	✗	✓
Integrated observability	Limited	Limited	✓
Configuration Complexity	Low	Low	High
CNI independence	✓	✗	✓
Traffic management	✗	✗	✓
Multicloud support	✓	✓	✓
End user persona	Developer	Cluster admin	Cluster admin, Developer
License	Apache 2.0	Apache 2.0	Apache 2.0
CNCF hosted	✗	✓	✓

Testbed


Kubernetes Cluster 1
Control Plane
Gateway Node
Data-Plane Node
Control Plane
Gateway Node
Data-Plane Node
Kubernetes Cluster 2
SUT


OpenShift

Performance profile

```yaml
apiVersion: performance.openshift.io/v2
kind: PerformanceProfile
metadata:
  name: perf-profile
spec:
  cpu:
    isolated: 4-39,44-79
    reserved: 0-3,40-43
  globallyDisableIrqLoadBalancing: false
  workloadHints:
    realTime: false
    highPowerConsumption: false
    perPodPowerManagement: true
  realTimeKernel:
    enabled: false
  numa:
    topologyPolicy: "single-numa-node"
  net:
    userLevelNetworking: false
```

Pod definition

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: client-pp
  namespace: default
spec:
  template:
    metadata:
      annotations:
        irq-load-balancing.crio.io: "disable"
        cpu-quota.crio.io: "disable"
    spec:
      runtimeClassName: performance-perf-profile
```

Tooling

- Layer 4: uperf (stream and request-response)
- Layer 7: wrk2
- Energy monitoring
  - Kubernetes Efficient Power Level Exporter (Kepler)
  - BCM

Assets can be found at:
https://github.com/RedHatResearch/icpe25-multi-cluster-networking

Uperf - TCP_STREAM

Sends as many write operations as possible during 1m

```
<profile name="throughput">
     <group nthreads="$nthr">
         <transaction iterations="1">
             <flowop type="connect" options="remotehost=$uperf_service_host protocol=tcp port=$uperf_service_port_uperf_data"/>
         </transaction>
         <transaction duration="60s">
             <flowop type="write" options="size=$size"/>
         </transaction>
         <transaction iterations="1">
             <flowop type="disconnect" />
         </transaction>
     </group>
   </profile>
```

Uperf - TCP_RR

Get latency by measuring the duration of small write/read operations during 1 min

```
<profile name="rr">
   <group nthreads="$nthr">
       <transaction iterations="1">
           <flowop type="connect" options="remotehost=$uperf_service_host protocol=tcp port=$uperf_service_port_uperf_data"/>
       </transaction>
       <transaction duration="60s">
           <flowop type="write" options="size=$size"/>
           <flowop type="read" options="size=$size"/>
       </transaction>
       <transaction iterations="1">
           <flowop type="disconnect" />
       </transaction>
   </group>
</profile>
```

Power monitoring: Kepler

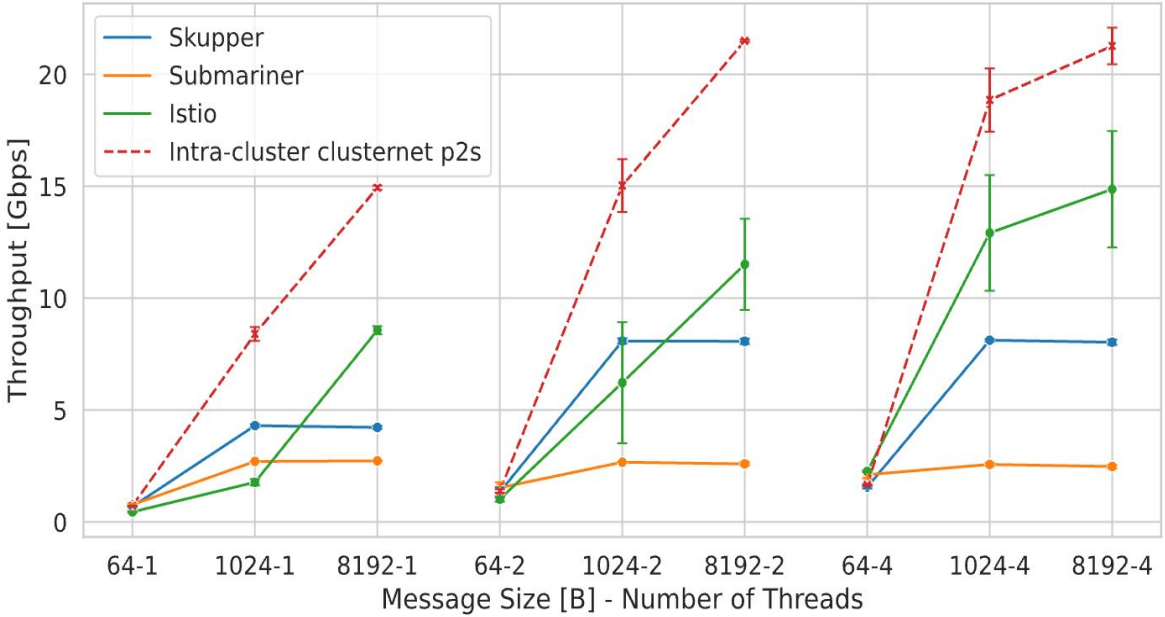
Exposes power monitoring Prometheus metrics.

Runs a DaemonSet that collects real-time CPU power consumption data from nodes:

- **RAPL** (Running average power limit Intel/AMD): High-precision monitoring of CPU package and DRAM energy usage.
- **Redfish**: Real-time platform power usage via the Baseboard Management Controller (BMC).

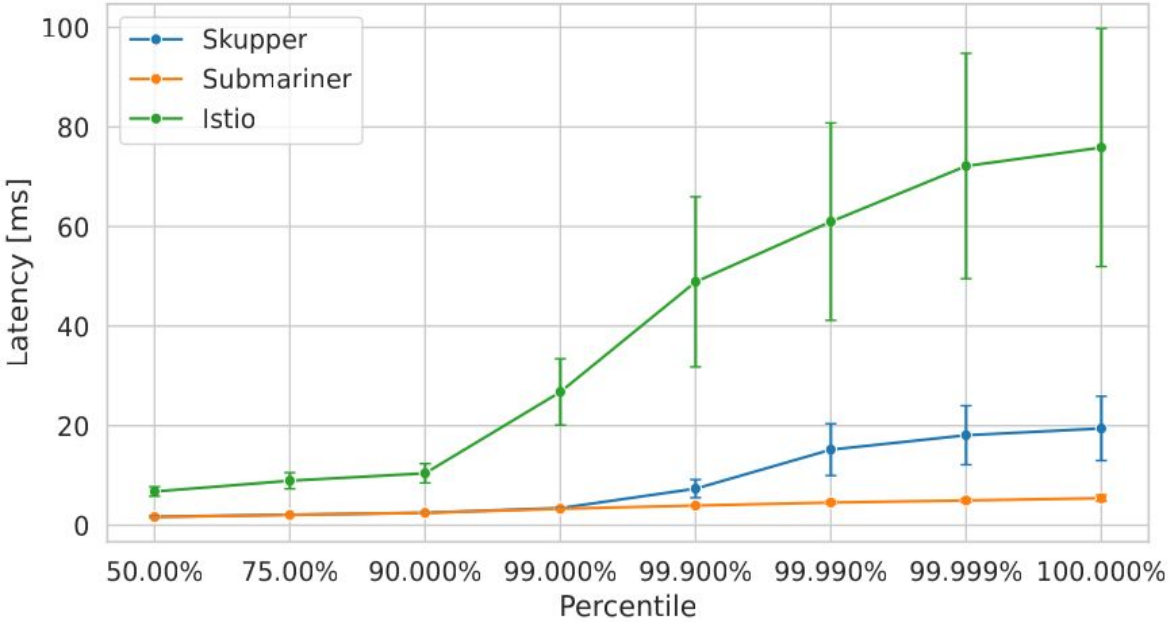
Redfish was our choice as we wanted to measure the total power footprint including non-compute components. (NICs, cooling fans, power supply overhead, etc.)

Throughput - TCP_STREAM



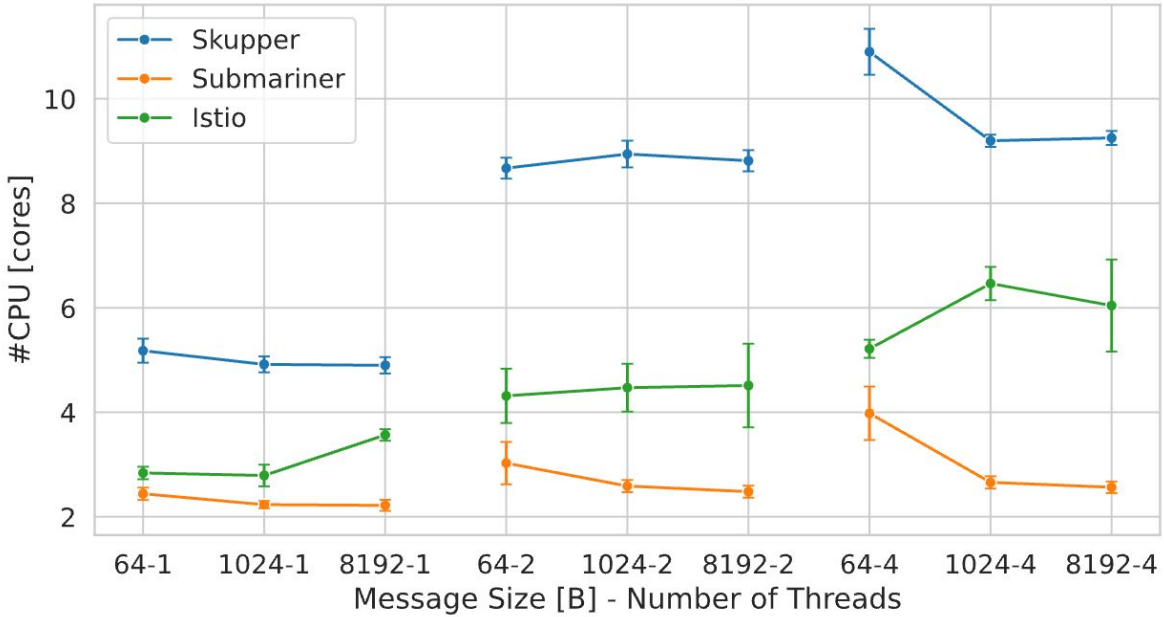
- **Highest throughput with Istio, Skupper** is limited at **~8Gbps** and **Submariner** comes in as the lowest at **~3Gbps** due to a IPSec/Kernel implementation limitation.
- RFC 9611 introduces support for encrypted traffic to be handled across multiple CPUs without splitting logical tunnels

HTTP - latency



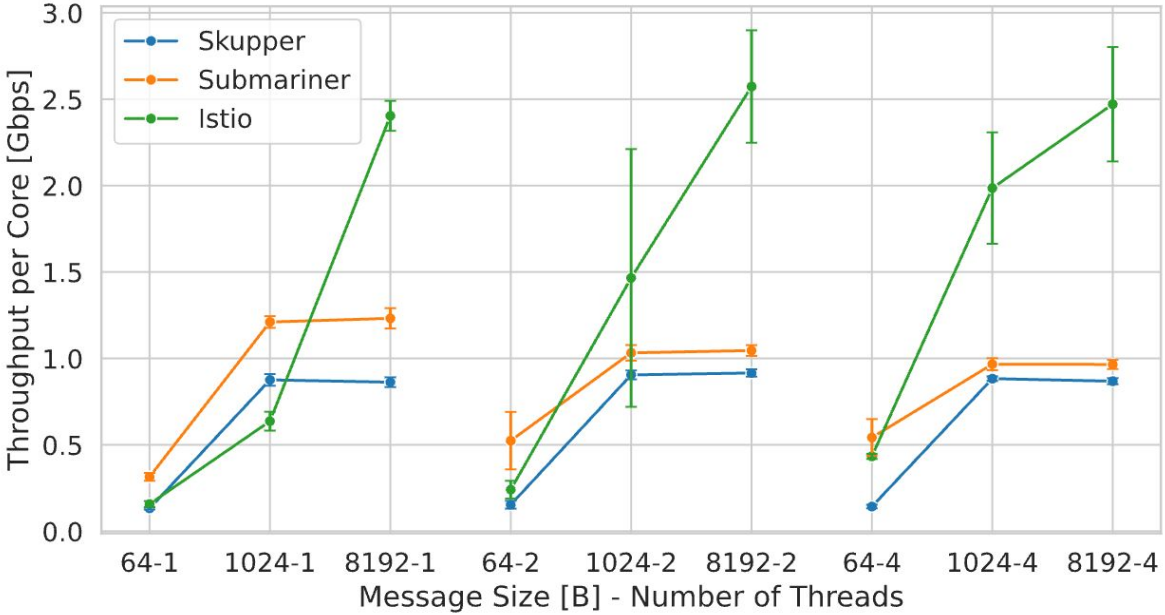
- 10K RPS, 100 connections, 2 threads, 1 KiB body size
- Significant latency overhead in Istio multi-cluster

Resource consumption overhead



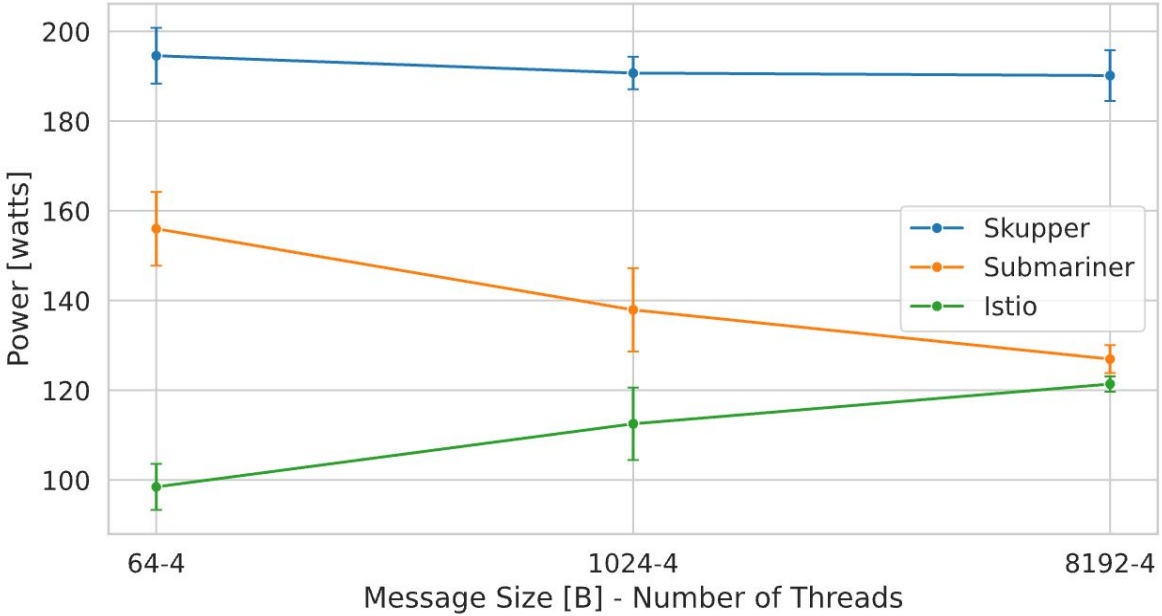
- **Submariner is the most resource efficient**, followed by Istio
- Skupper consumes the most number of CPUs at all packet size/thread combinations

Normalized resource consumption



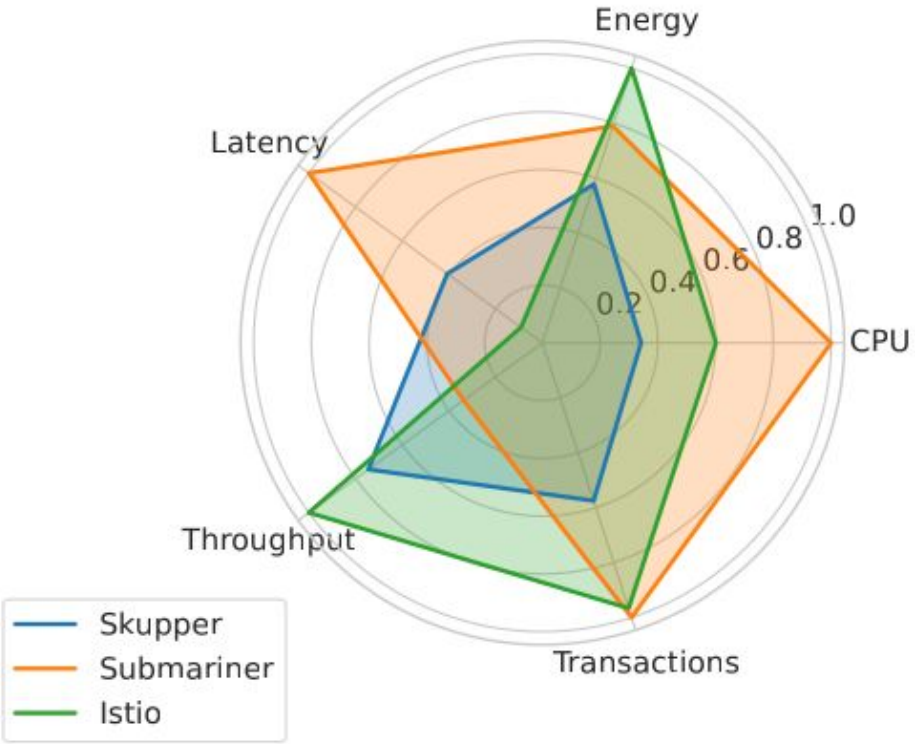
Istio and Submariner do better than Skupper on **throughput/core**

Power consumption



Higher **CPU usage** is correlated with higher power consumption

Summary


Energy
Latency
CPU
Throughput
Transactions
0.2
0.4
0.6
0.8
1.0
Skupper
Submariner
Istio

Future work

- Leverage **real-world benchmarks** involving messaging, databases and applications based on microservices architecture
- Explore are key capabilities of the **control plane**
  - Service discovery time
  - Service failure time
- Scale the **number of clusters** under test
- Include **Istio ambient mode** in the test matrix

Thank you!